UNIVERSIDAD PRIVADA
DOMINGO SAVIO

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Facultad de Ingeniería

ACTIVIDAD 3
USUARIOS, ROLES Y REPORTES

Apicultores Flor de la Chiquitanía

Autenticación con roles, operaciones transaccionales y reportes al sistema de la asociación

Integrantes:

1. Daniela Cayo Mamani
2. Juan Jose Vicente Cossio
3. Ronald Salvatierra Flores
4. Yordy Tolavi Ruiz
5. Alejandro Cadario Lopez

Carrera: Ingeniería de Sistemas

Asignatura: Tecnología Web II

Docente: M.Sc. Ing. Jimmy Nataniel Requena Llorentty

Santa Cruz de la Sierra, Bolivia

20 de octubre de 2026

Ing. Jimmy Requena Llorentty
INGENIERO DE SISTEMAS
AIGENTIC BOLIVIANOTECH

Resumen

Este informe documenta la incorporación de autenticación con roles, operaciones transaccionales y reportes al sistema web de la Asociación de Apicultores Flor de la Chiquitanía. A la aplicación en PHP y MySQL se agregaron una tabla de usuarios con contraseñas protegidas mediante password_hash, el registro de usuarios por parte de la administración, un inicio de sesión que regenera el identificador de sesión, un control de acceso por rol aplicado en el servidor, tokens CSRF en todos los formularios y escape de salida contra XSS. Se añadieron además el registro de ventas y acopios con transacciones y actualización atómica del stock, y cuatro reportes con JOIN, GROUP BY, HAVING y LEFT JOIN exportables a CSV, con neutralización de fórmulas. El acceso de cada rol se comprobó con 56 peticiones HTTP reales (14 rutas por 4 perfiles) y con un verificador automático de 17 comprobaciones; los verificadores de los laboratorios 10 y 11 dieron 15 de 15 en Docker. El código se publica en un repositorio de GitHub.

Palabras clave: autenticación, control de acceso por roles, CSRF, transacciones, reportes, CSV, PHP, MySQL

Autenticación con roles, operaciones transaccionales y reportes para la asociación de apicultores

Introducción

La asociación gestiona su catálogo de productos y a sus socios productores con una aplicación web que, hasta la actividad anterior, guardaba los datos en MySQL con consultas preparadas pero no distinguía quién usaba el sistema: cualquier persona con la dirección podía ver, modificar o eliminar registros. Una asociación que vende, recibe producto de sus socios y rinde cuentas necesita saber quién hace cada operación y limitar lo que cada persona puede hacer. Esta actividad resuelve ese problema en tres frentes: identificar a las personas (autenticación), limitar sus acciones según su función (autorización por roles) y registrar las operaciones del negocio con garantías de consistencia (transacciones y reportes).

El trabajo integra los laboratorios 10 y 11 de la asignatura (Requena Llorentty, 2026) y agrega lo que la actividad pide y los laboratorios no incluyen: el registro de usuarios y una prueba sistemática de lo que puede y no puede hacer cada rol. El informe describe el modelo de datos, las defensas implementadas, las pruebas realizadas y sus resultados. El código está disponible en el repositorio del grupo (Cayo Mamani et al., 2026), cuyo enlace se repite en el Anexo A.

Objetivos

Objetivo general

Agregar autenticación con roles, operaciones transaccionales y reportes al sistema de la asociación y demostrar que cada rol accede solo a lo que le corresponde.

Objetivos específicos

- Almacenar usuarios con contraseñas protegidas con password_hash y permitir su registro e inicio de sesión con password_verify y regeneración del identificador de sesión.
- Controlar el acceso por rol (administración, operador y socio) en el servidor, no solo ocultando botones.
- Proteger todos los formularios con tokens CSRF y escapar toda salida para evitar XSS.
- Registrar ventas y acopios con transacciones y actualización de stock sin sobreventa.
- Generar reportes con JOIN y GROUP BY y exportarlos a CSV sin permitir inyección de fórmulas.
- Probar los accesos de cada rol y documentar el uso de inteligencia artificial en una bitácora.

Entorno y herramientas

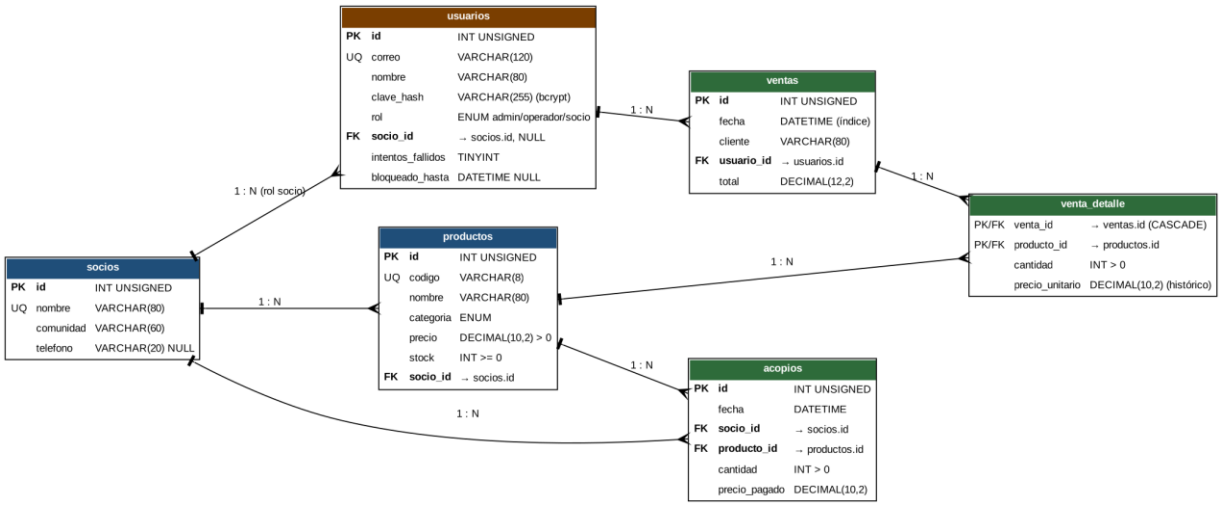
La aplicación se ejecuta con Docker Compose: un contenedor con MySQL 8.4 (Oracle Corporation, s. f.-b) y otro con PHP 8.4, con el código en un volumen. Docker Desktop corre sobre WSL 2 en Windows 10 (Docker, Inc., s. f.). La conexión a la base de datos se configura con variables de entorno, fuera del directorio público. Las dependencias se gestionan con Composer, con carga automática PSR-4 del espacio de nombres Apicultores. Para las pruebas se usó un verificador propio (Verificador.php) que levanta el servidor PHP y envía peticiones HTTP reales con cookies, como lo haría un navegador. Las capturas provienen del equipo de un integrante del grupo; las demás pruebas se ejecutaron además en un entorno de preparación con PHP 8.3 y MariaDB 10.11.

Modelo de datos

Sobre las tablas socios y productos (relación 1:N) se agregaron cuatro tablas (Figura 1): usuarios, con la contraseña solo como hash; ventas y venta_detalle, que forman una relación de uno a muchos con clave primaria compuesta; y acopios, donde cada socio entrega producto a la asociación. La Tabla 1 resume las decisiones de diseño que sostienen la integridad de los datos.

Figura 1

Modelo entidad-relación de la base de datos apicultores



Nota. Elaboración propia a partir de sql/01-esquema.sql, sql/04-usuarios.sql y sql/05-movimientos.sql. Las tablas de color verde son las del laboratorio 11 y la marrón la del laboratorio 10.

Tabla 1

Decisiones de diseño del modelo

Elemento	Decisión	Para qué
usuarios.clave_hash	VARCHAR(255)	Los algoritmos futuros generan hashes más largos que bcrypt.
usuarios.correo	Índice único	Un correo identifica a una sola persona; la búsqueda es exacta.
usuarios.rol	ENUM(admin, operador, socio)	Solo roles válidos; la matriz de permisos vive en el código.
usuarios.intentos_fallidos, bloqueado_hasta	Contador y fecha	Bloqueo de la cuenta tras cinco contraseñas erróneas.
venta_detalle (venta_id, producto_id)	Clave primaria compuesta	Un producto aparece una sola vez por venta.

Elemento	Decisión	Para qué
venta_detalle.precio_unitario	Precio del día de la venta	Los reportes no cambian cuando cambia el precio del producto.
ventas.fecha	Índice ix_ventas_fecha	Los reportes filtran siempre por rango de fechas.
productos.stock	CHECK (stock >= 0)	El stock nunca queda negativo, ni por error de programa.
Claves foráneas	ON DELETE RESTRICT; CASCADE solo en venta_detalle → ventas	No se borran socios ni productos con movimientos.

Nota. Fuente: sql/ del repositorio.

Autenticación y sesiones

Contraseñas

Las contraseñas nunca se guardan en texto: se convierten con password_hash (algoritmo bcrypt, costo 12), que genera una sal aleatoria para cada contraseña y la incluye dentro del propio hash (PHP Group, s. f.-a; OWASP Foundation, s. f.-a). Al iniciar sesión se compara con password_verify. Como el operador de prueba tiene un hash de costo 10, el sistema aplica password_needs_rehash y lo actualiza a costo 12 en el momento del ingreso, que es el único instante en que se dispone de la contraseña en texto. El verificador confirma que ninguna contraseña aparece en la tabla y que el hash del operador sube de $2y$10$ a $2y$12$.

Registro de usuarios

El registro de usuarios es una función de la administración (/usuarios/nuevo). Se decidió no ofrecer un registro público porque el sistema es interno de la asociación: un registro abierto permitiría que cualquier persona obtuviera una cuenta y, con ella, acceso a datos de ventas y socios. El administrador asigna el rol y, para el rol socio, vincula al usuario con un socio existente; ese vínculo es el que luego limita qué productos ve. La Tabla 2 resume las validaciones, que se aplican en el servidor.

Tabla 2

Validaciones del registro de usuarios

Campo	Regla	Respuesta si falla
Correo	Formato válido, hasta 120 caracteres, único; se guarda en minúsculas	422 con el mensaje en el campo
Nombre	Entre 3 y 80 caracteres	422
Contraseña	Al menos 10 caracteres con letras y números; repetida igual; sin contener la parte local del correo	422
Rol	Uno de la lista (admin, operador, socio)	422
Socio vinculado	Obligatorio y existente si el rol es socio; se ignora en los demás roles	422
Token CSRF y permiso	Solo el rol administración puede registrar	403

Nota. Fuente: src/Validacion/ValidadorUsuario.php. La contraseña nunca se vuelve a mostrar en el formulario.

Inicio de sesión y sesión

El formulario de ingreso (Figura 2) responde 401 con el mismo mensaje cuando el correo no existe y cuando la contraseña es incorrecta, para no revelar qué correos están registrados; si el correo no existe, además se verifica un hash ficticio para que el tiempo de respuesta sea parecido. Tras cinco fallos consecutivos la cuenta se bloquea durante quince minutos y la siguiente solicitud, aunque sea correcta, recibe 429 con la cabecera Retry-After. Al ingresar correctamente se llama a session_regenerate_id(true), de modo que un identificador de sesión conocido por un atacante antes del ingreso (fijación de sesión) deja de servir (PHP Group, s. f.-c; OWASP Foundation, s. f.-d).

Figura 2

Pantalla de inicio de sesión



Nota. Captura propia de http://127.0.0.1:8000/entrar.

La sesión guarda únicamente el identificador, el correo, el nombre, el rol y el socio vinculado; nunca la contraseña ni su hash. La Tabla 3 resume la configuración de la cookie, y a los 30 minutos sin actividad la sesión se cierra.

Tabla 3

Configuración de la sesión

Opción	Valor	Ataque que mitiga
Nombre de la cookie	APISESION	No anuncia que se usa PHP (PHPSESSID).
HttpOnly	Sí	JavaScript no puede leer la cookie: un XSS no roba la sesión.
SameSite	Lax	El navegador no envía la cookie en POST iniciados desde otro sitio (CSRF).
Secure	Solo bajo HTTPS	La cookie no viaja en claro por redes abiertas.
use_strict_mode y use_only_cookies	Activados	Rechaza identificadores no creados por el servidor; el ID nunca va en la URL.
Inactividad	1800 s	Una sesión olvidada no queda abierta indefinidamente.

Nota. Fuente: src/Seguridad/Sesion.php.

Protección contra CSRF y XSS

Todo formulario que cambia el estado del sistema (ingreso, salida, registro y edición de productos, eliminación, ventas, acopios y registro de usuarios) incluye un campo oculto con un token generado en la sesión con random_bytes. El front controller valida el token en cada petición POST con hash_equals, que compara en tiempo constante y evita ataques de

temporización; si falta o no coincide, responde 403 (OWASP Foundation, s. f.-b). Las peticiones GET no modifican datos: pedir la ruta de eliminar con GET responde 405. SameSite=Lax es una segunda capa independiente, no un sustituto del token.

Contra XSS, el motor de plantillas escapa automáticamente con htmlspecialchars todo texto que llega a la vista, de modo que olvidar escapar deja de ser posible; la política de seguridad de contenido (Content-Security-Policy) agrega otra capa (OWASP Foundation, s. f.-c). Se comprobó con el texto <script>alert(1)</script> en el nombre de un producto, en el nombre de un usuario y en la búsqueda: en los tres casos se imprime como texto. La Tabla 4 relaciona cada ataque con su defensa.

Tabla 4

Ataques considerados, defensa y línea de código

Ataque	**Defensa**	**Código**
Robo de contraseñas por filtración de la base	Hash lento con sal	password_hash($clave, PASSWORD_DEFAULT, ['cost' => 12])
Robo de cookie	Cookie HttpOnly y Secure	'httponly' => true, 'secure' => $https
Fijación de sesión	ID nuevo al ingresar	session_regenerate_id(true);
CSRF	Token por sesión y SameSite	hash_equals($esperado, $recibido)
Fuerza bruta	Bloqueo tras 5 fallos (429)	throw new DemasiadosIntentos(...)
XSS	Escape automático de salida	htmlspecialchars($valor, ENT_QUOTES, 'UTF-8')
Enumeración de usuarios	Mismo mensaje y mismo tiempo	password_verify($clave, self::HASH_FICTICIO)
Redirección abierta (volver=//sitio)	Solo rutas internas	preg_match('#^/(?!/)[\w\-/?=&%.]*$#', $v)

Nota. Fuente: src/Seguridad, src/Vista y src/Controladores del repositorio.

Control de acceso por rol

Los permisos viven en un único lugar, la matriz de la clase Rol (Tabla 5), y cada acción del controlador la consulta antes de actuar. Una acción que no está en la matriz se niega por defecto. Ocultar un botón no basta, porque cualquiera puede enviar la petición a mano; por eso el servidor responde 403 aunque la URL se escriba directamente.

Tabla 5

Matriz de permisos por rol

Acción	Administración	Operador	Socio
Ver productos (el socio solo los suyos)	Sí	Sí	Sí
Registrar y editar productos	Sí	Sí	No
Eliminar productos	Sí	No	No
Ver socios	Sí	Sí	No
Registrar ventas y acopios	Sí	Sí	No
Reportes y exportación CSV	Sí	No	No
Registrar y listar usuarios	Sí	No	No

Nota. Fuente: src/Modelo/Rol.php.

Para demostrar que cada rol accede solo a lo que le corresponde, el script demo_roles.php envía 14 peticiones a cada uno de cuatro perfiles (anónimo, administración, operador y socio), 56 en total, y compara el estado HTTP con el esperado (Tabla 6). Las rutas POST se prueban con datos vacíos o recursos inexistentes: así no modifican datos y se distingue el 403 (el rol no tiene permiso) del 404 o 422 (el rol sí tiene permiso, pero la petición no tiene sentido). Las 56 respuestas coinciden con lo esperado.

Tabla 6

Respuesta HTTP de cada ruta según el perfil

Ruta	Acción	Anón.	Admin	Oper.	Socio
GET /productos	Ver productos	302	200	200	200
GET /productos/nuevo	Formulario de producto	302	200	200	403
POST /productos	Registrar producto (vacío)	302	422	422	403
POST /productos/ZZZ-999	Editar producto inexistente	302	404	404	403
POST /productos/ZZZ-999/eliminar	Eliminar producto inexistente	302	404	403	403
GET /socios	Ver socios	302	200	200	403
GET /ventas/nueva	Formulario de venta y acopio	302	200	200	403
POST /ventas	Registrar venta (vacía)	302	422	422	403
POST /acopios	Registrar acopio (vacío)	302	422	422	403
GET /reportes	Reportes	302	200	403	403
GET /reportes/ventas.csv	Exportar CSV	302	200	403	403

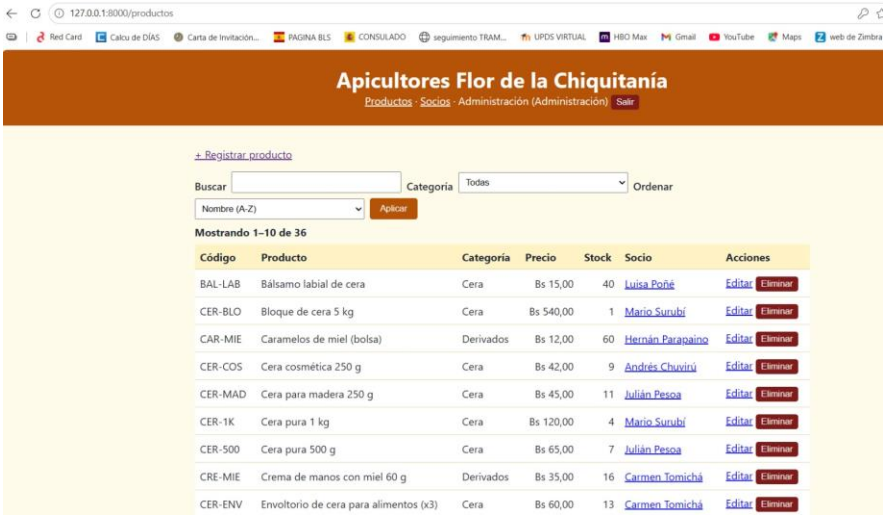
Ruta	Acción	Anón.	Admin	Oper.	Socio
GET /usuarios	Lista de usuarios	302	200	403	403
GET /usuarios/nuevo	Formulario de usuario	302	200	403	403
POST /usuarios	Registrar usuario (vacío)	302	422	403	403

Nota. Salida de demo_roles.php. 302: redirige a iniciar sesión; 403: el rol no tiene permiso; 404 y 422 en rutas POST: el rol tiene permiso. Ejecutado en el entorno de preparación (PHP 8.3 y MariaDB 10.11).

Las capturas del navegador muestran la misma diferencia. La administración ve Editar y Eliminar en cada producto (Figura 3); el operador ve Registrar producto y Editar, pero no Eliminar, y su menú no incluye Reportes ni Usuarios (Figura 4); y la socia Elena ve solo sus cuatro productos, sin acciones y sin acceso a ninguna otra sección (Figura 5).

Figura 3

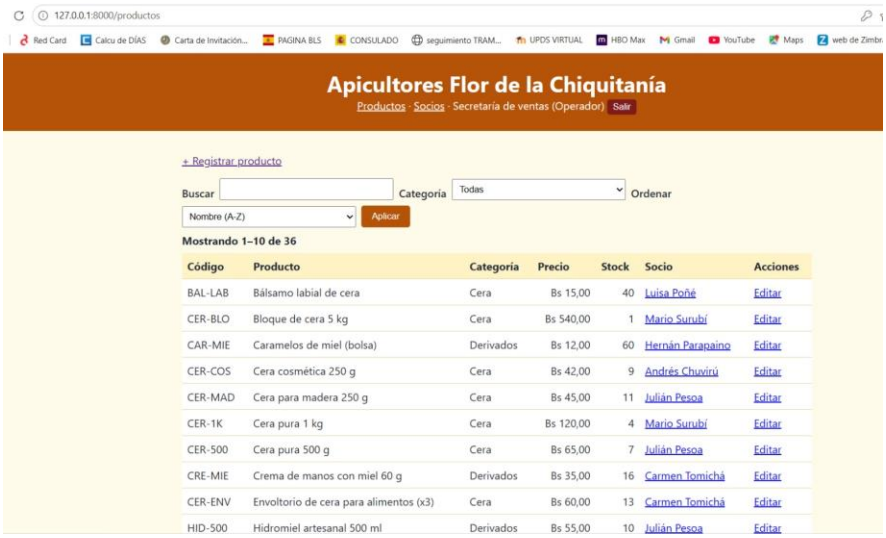
Administración: lista de productos con Editar y Eliminar



Nota. Captura propia de http://127.0.0.1:8000/productos, con sesión de administración.

Figura 4

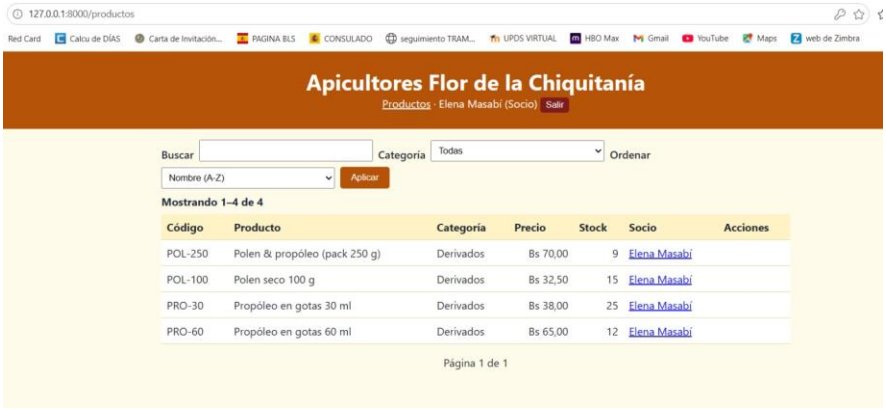
Operador: puede registrar y editar, no eliminar



Nota. Captura propia con sesión de operador. El menú no muestra Reportes ni Usuarios.

Figura 5

Socia: solo ve sus propios productos



Nota. Captura propia con sesión de Elena Masabí: «Mostrando 1–4 de 4». El filtro se aplica en la consulta (socio_id del usuario), no en la plantilla.

Operaciones transaccionales

Ventas

Una venta de varios productos es una sola operación: se descuenta el stock de cada línea, se crea la venta y se guardan sus detalles dentro de una transacción (beginTransaction, commit y

rollBack en caso de error). El descuento usa una sola sentencia atómica: UPDATE productos SET stock = stock - :cantidad WHERE id = :id AND stock >= :minimo. Si no se modifica ninguna fila, el stock no alcanza y se lanza una excepción de conflicto, que la aplicación responde con 409 y no con 500, porque la petición es válida pero choca con el estado de los datos. Leer el stock con un SELECT, compararlo en PHP y luego actualizar dejaría una ventana en la que dos vendedoras a la vez venderían las últimas unidades y dejarían el stock negativo.

La prueba se hizo con una venta de 2 unidades de MIE-500 y 99 de CER-1K, cuando de CER-1K quedan 4. La aplicación respondió 409 (Figura 6) y el stock quedó igual que antes, MIE-500 = 32, CER-1K = 4 y PRO-30 = 25, con 96 ventas (Figura 7): el descuento de la primera línea se deshizo. Después se registró una venta válida de 2 MIE-500 y 1 PRO-30 por Bs 128,00 (Figura 8) y el stock pasó a 30 y 24 con 97 ventas (Figura 9). Cada línea guarda el precio unitario de ese día.

Figura 6

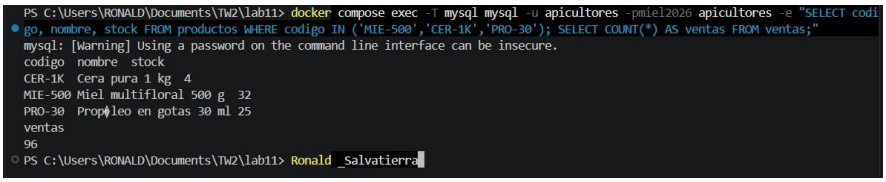
Venta rechazada por stock insuficiente (409)



Nota. Captura propia: «Stock insuficiente de CER-1K para vender 99».

Figura 7

Stock y ventas tras el rollback



Nota. Captura propia de la consola de MySQL: los valores no cambiaron (32, 4 y 25; 96 ventas).

Figura 8

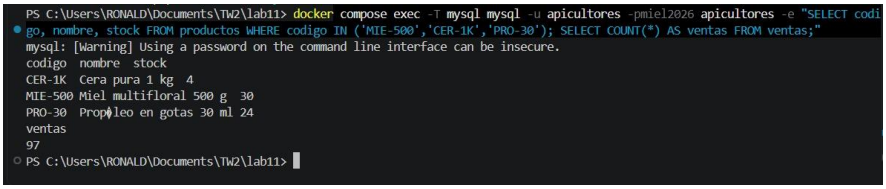
Venta válida registrada



Nota. Captura propia: «Venta N.º 97 registrada» (2 MIE-500 y 1 PRO-30).

Figura 9

Stock y ventas después de la venta válida



Nota. Captura propia: MIE-500 = 30, PRO-30 = 24 y 97 ventas.

Acopios

Un acopio registra que un socio entrega producto a la asociación: aumenta el stock y guarda el socio dueño del producto, la fecha, la cantidad y el precio pagado. Se ejecuta en una transacción que primero lee la fila del producto con SELECT … FOR UPDATE, que la bloquea

hasta el final de la transacción para que nadie la modifique entre la lectura y la actualización (Oracle Corporation, s. f.-a). Una cantidad de cero se rechaza con 422 y no altera el stock. El verificador comprueba que un acopio de 10 unidades de CER-1K sube el stock de 4 a 14 y se asocia al socio correcto.

Reportes con JOIN y GROUP BY y exportación a CSV

Solo la administración accede a los reportes (Figura 10). La Tabla 7 resume las cuatro consultas, todas preparadas y con el período recibido como parámetro. El período usa fecha < hasta + 1 día y no fecha <= hasta, porque fecha es de tipo DATETIME y, con la segunda forma, las ventas del último día después de las 00:00 quedarían fuera.

Tabla 7

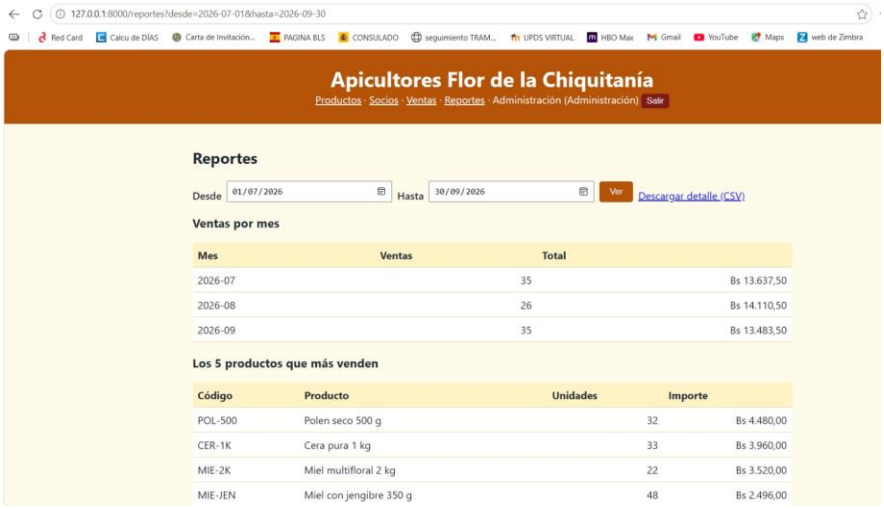
Reportes y técnicas de SQL aplicadas

Reporte	Técnica	Resultado (jul–sep 2026)
Ventas por mes	COUNT y SUM con GROUP BY por mes	Bs 13.637,50; 14.110,50 y 13.483,50
Cinco productos más vendidos	JOIN de tres tablas, SUM, ORDER BY y LIMIT	POL-500, CER-1K, MIE-2K, MIE-JEN, MIE-TOC
Socios con Bs 3.000 o más	JOIN de cuatro tablas, GROUP BY y HAVING	4 de 8 socios
Productos sin ventas	LEFT JOIN … IS NULL	CER-BLO, JAL-30, MIE-OR, VIN-MIE

Nota. Fuente: src/Reportes/ReporteVentas.php. El verificador del laboratorio 11 compara cada reporte con un cálculo independiente hecho en PHP sobre las filas sin agrupar.

Figura 10

Pantalla de reportes (administración)



Nota. Captura propia de http://127.0.0.1:8000/reportes: ventas por mes y productos más vendidos.

La exportación a CSV (Tabla 8) usa una línea por detalle de venta. Un cliente cuyo nombre empiece por =, +, - o @ se interpretaría como fórmula al abrir el archivo en una hoja de cálculo (inyección CSV); por eso la función celdaSegura antepone un apóstrofo a esos textos (OWASP Foundation, s. f.-e). El verificador registra una venta con el cliente =HYPERLINK("http://ejemplo.invalid","Ver factura") y comprueba que en el CSV aparece como texto.

Tabla 8

Características del archivo CSV exportado

Característica	Valor	Motivo
Content-Type y descarga	text/csv; charset=utf-8; attachment	El navegador descarga el archivo en lugar de mostrarlo.
BOM UTF-8	EF BB BF al inicio	Excel reconoce UTF-8 y muestra bien tildes y ñ.
Separador	Punto y coma, con coma decimal	Es lo que Excel espera en configuración regional española (Shafranovich, 2005).
Filas	Una por detalle: 170 más el encabezado en el trimestre	Permite sumar y filtrar en la hoja de cálculo.
Fórmulas	Apóstrofo antepuesto a =, +, - y @	Evita la inyección de fórmulas.

Nota. Fuente: src/Reportes/Csv.php y el archivo ventas_20260701_20260930.csv descargado.

Pruebas y resultados

Se usaron tres verificadores automáticos (Tabla 9). Los de los laboratorios 10 y 11 fueron ejecutados en Docker por un integrante del grupo y dieron 15 de 15 cada uno (Figuras 11 y 12). El verificador de la actividad, verificar_actividad3.php, reúne 17 comprobaciones: tabla de usuarios y hash, inicio de sesión con ID nuevo, registro de usuarios y sus rechazos, bloqueo por intentos, CSRF en todos los formularios, XSS, la matriz de 56 accesos, los permisos del operador y de la socia, la venta con rollback, la venta correcta, el acopio, los reportes y el CSV. Dio 17 de 17 en el entorno de preparación (PHP 8.3 y MariaDB 10.11), y su ejecución en Docker con MySQL 8.4 se anexará al repositorio como evidencia.

Tabla 9

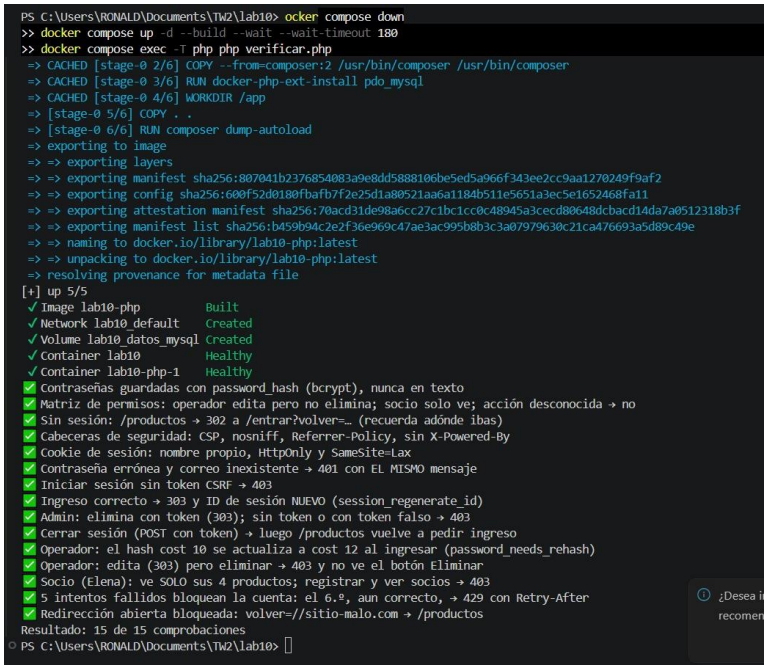
Resultados de los verificadores

Verificador	Alcance	Resultado
Laboratorio 10	Contraseñas, sesión, CSRF, roles, bloqueo y redirecciones	15 de 15 (Docker)
Laboratorio 11	Transacciones, acopios, reportes y CSV	15 de 15 (Docker)
Actividad 3	Integración, registro de usuarios y matriz de accesos	17 de 17 (preparación)

Nota. Fuente: salida de cada verificador.

Figura 11

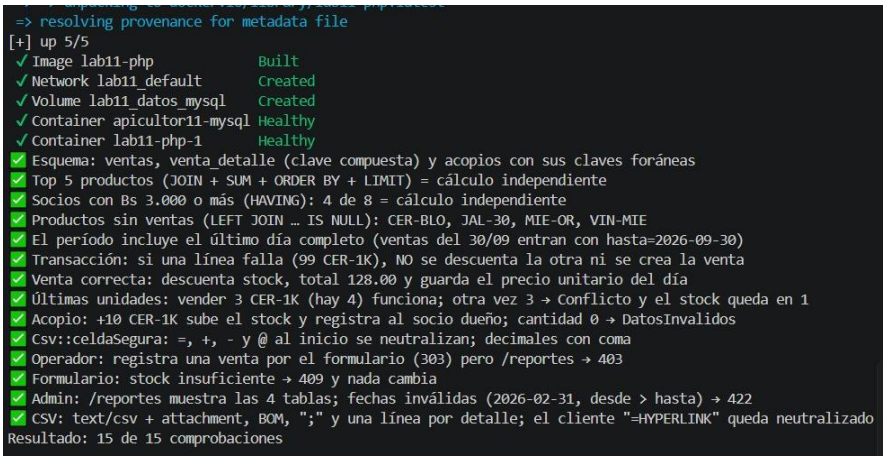
Verificador del laboratorio 10



Nota. Captura propia: 15 de 15 comprobaciones.

Figura 12

Verificador del laboratorio 11



Nota. Captura propia: 15 de 15 comprobaciones.

Para comprobar que las pruebas detectan fallas reales se hicieron pruebas de mutación sobre una copia del código: (a) dar al operador permiso para eliminar productos hizo fallar la

matriz de accesos y la prueba del operador (15 de 17); (b) quitar la validación del token CSRF hizo fallar la comprobación de CSRF; (c) desactivar el escape de salida hizo fallar la de XSS; y (d) quitar session_regenerate_id hizo fallar la de inicio de sesión. En cada caso el verificador pasó de 17 a 15 o 16 comprobaciones superadas, y con el código correcto vuelve a 17.

Dificultades y soluciones

Tabla 10

Problemas encontrados y cómo se resolvieron

Problema	Causa	Solución
«no configuration file provided»	Comandos de Docker ejecutados en la carpeta equivocada	Entrar a la carpeta del proyecto, donde está docker-compose.yml.
El compose del laboratorio 11 no tenía PHP	Solo definía el servicio mysql	Se agregó el servicio php y su Dockerfile, y name: del proyecto.
Se vendió todo el stock en una prueba	Se escribieron los stocks como cantidades	Se reinició la base con down -v y up, y se repitió con 2 y 1 unidades.
CHECK de MySQL 8.4 devuelve HY000 (3819)	Un verificador esperaba el código SQLSTATE 23000	Comprobar el error sin depender del SQLSTATE.
Volumen o puertos compartidos entre laboratorios	Mismo nombre de proyecto y puertos 8000 y 3307	name: propio por proyecto y detener el anterior con down.

Nota. Fuente: registro de trabajo del grupo.

Limitaciones

La protección actual no cubre todo. La cookie solo lleva Secure cuando hay HTTPS, y el entorno de laboratorio usa HTTP; en producción debe publicarse con HTTPS. El bloqueo por intentos es por cuenta y no por dirección de origen, de modo que un atacante podría bloquear la cuenta de otra persona; en producción convendría combinarlo con límites por dirección. Los permisos están en el código y no en una tabla editable. Las contraseñas de demostración están a la vista en la documentación y no deben usarse fuera del laboratorio. Estas mejoras quedan para actividades posteriores.

Uso de inteligencia artificial

Durante el proyecto se utilizó el modelo de lenguaje Claude (Anthropic, 2026) como asistente para revisar los códigos de partida, completar los TODO de los laboratorios, escribir verificadores, explicar errores y redactar borradores de las hojas de trabajo y de este informe. Cada uso está registrado en la bitácora de IA del proyecto (Bitacora_IA_Proyecto), con la solicitud, el aporte y la verificación realizada. Los resultados se comprobaron ejecutando los verificadores y las pruebas en el equipo de un integrante del grupo, y las explicaciones conceptuales se contrastaron con el código y con las capturas propias.

Conclusiones

El sistema de la asociación ahora sabe quién lo usa y limita lo que cada persona puede hacer. Las contraseñas se guardan solo como hash, la sesión se regenera al ingresar, los formularios llevan token CSRF y la salida se escapa; el acceso por rol se aplica en el servidor y se demostró con 56 peticiones reales, donde el operador no puede eliminar ni ver reportes, y la socia solo ve sus productos. Las ventas y los acopios son operaciones atómicas que mantienen el stock consistente y no permiten sobreventa, y los reportes responden preguntas de la directiva con JOIN, GROUP BY y HAVING y se exportan a CSV sin riesgo de fórmulas. Las pruebas automáticas, que fallan cuando se introduce un defecto a propósito, dan confianza en que esas garantías se mantendrán al seguir desarrollando el sistema.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Anthropic. (2026). *Claude* (Sonnet 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai

Cayo Mamani, D., Vicente Cossio, J. J., Salvatierra Flores, R., Tolavi Ruiz, Y., & Cadario Lopez, A. (2026). *Actividad03_tw2* [Código fuente]. GitHub. https://github.com/RonSalvet/Actividad03_tw2.git

Docker, Inc. (s. f.). *Docker Compose overview*. https://docs.docker.com/compose/

Oracle Corporation. (s. f.-a). *MySQL 8.4 reference manual: Locking reads*. https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

Oracle Corporation. (s. f.-b). *MySQL 8.4 reference manual*. https://dev.mysql.com/doc/refman/8.4/en/

OWASP Foundation. (s. f.-a). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. (s. f.-b). *Cross-site request forgery prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP Foundation. (s. f.-c). *Cross site scripting prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

OWASP Foundation. (s. f.-d). *Session management cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

OWASP Foundation. (s. f.-e). *CSV injection*. https://owasp.org/www-community/attacks/CSV_Injection

PHP Group. (s. f.-a). *password_hash*. En *PHP manual*. https://www.php.net/manual/en/function.password-hash.php

PHP Group. (s. f.-b). *PDO::prepare*. En *PHP manual*. https://www.php.net/manual/en/pdo.prepare.php

PHP Group. (s. f.-c). *session_regenerate_id*. En *PHP manual*. https://www.php.net/manual/en/function.session-regenerate-id.php

Requena Llorentty, J. N. (2026). *Guías didácticas de los laboratorios 10 y 11 de Tecnología Web II* [Material de curso]. Universidad Privada Domingo Savio.

Shafranovich, Y. (2005). *Common format and MIME type for comma-separated values (CSV) files* (RFC 4180). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc4180

Anexo A

Repositorio y cómo ejecutar el proyecto

Repositorio: https://github.com/RonSalvet/Actividad03_tw2.git

```
Actividad03_tw2/
├── actividad03/              Aplicación final (labs 10 y 11 + registro de usuarios)
│   ├── config/bd.php         Conexión por variables de entorno (fuera de public/)
│   ├── sql/                  Esquema, datos, usuarios, ventas y acopios
│   ├── src/                  Seguridad, roles, ventas, reportes y validación
│   ├── public/index.php      Front controller (cabeceras, sesión, CSRF, rutas)
│   ├── verificar_actividad3.php  17 comprobaciones
│   └── demo_roles.php        Matriz de accesos 14 rutas × 4 perfiles
├── laboratorios/lab10, lab11 Laboratorios con los TODO resueltos
└── docs/                     Informe, bitácora de IA, presentación y evidencias
```

Comandos (PowerShell, dentro de actividad03):

```
docker compose up -d --build --wait --wait-timeout 180
docker compose exec -T php php verificar_actividad3.php   # 17 comprobaciones
docker compose exec -T php php demo_roles.php             # matriz de accesos
# Sitio: http://127.0.0.1:8000
# Usuarios de demostración: admin@, operador@ y elena@florchiquitania.bo
```